

## READYKIT · DESIGN PRECIS

# ReadyKit design precis

Open Hardware Portfolio · amishchadha.com

## DOCUMENT CONTROL

|             |                                            |         |                |
|-------------|--------------------------------------------|---------|----------------|
| Document ID | RDK-PRC-001                                | Version | 0.2            |
| Status      | <b>Draft</b>                               | Date    | 2026-09-25     |
| Author      | Amish Chadha                               | License | CERN-OHL-S-2.0 |
| Project TRL | <b>TRL 2</b> Technology concept formulated |         |                |
| Repository  | github.com/BoujeeEnjinia1701/readykit      |         |                |
| Source      | docs/02-concept.md @ 5a3ae53               |         |                |

## REVISION HISTORY

| Version | Date       | Author       | Change                                                                                         |
|---------|------------|--------------|------------------------------------------------------------------------------------------------|
| 0.2     | 2026-09-25 | Amish Chadha | Architecture, modules, measured first-order numbers, design choices, safety and open questions |
| 0.1     | 2026-09-25 | Amish Chadha | Initial scaffold                                                                               |

# ReadyKit design precis

---

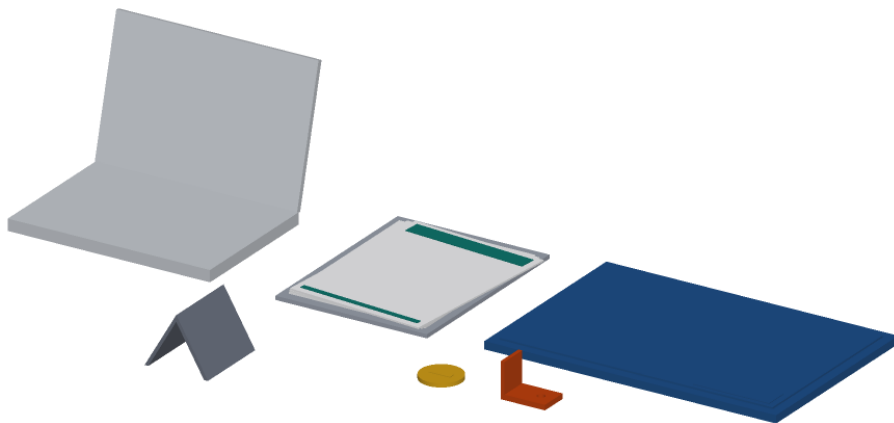
## ■ Summary

ReadyKit turns a Git repository of plain Markdown and build123d Python into checked, branded engineering documentation: controlled PDFs with revision history, drawing sheets, concept renders with a 3D viewer, and a Technology Readiness Level (TRL) that the checker will only accept when the evidence files exist. It already runs in every repository of the Design Molecule portfolio as the `.kit/` folder. The concept is to release it as a standalone open tool that any small hardware team can adopt in under 15 minutes.

Figure 1 shows an illustrative massing of the kit's outputs (`media/hero.png`); ReadyKit is a software project and has nothing to fabricate. Figure 2 shows the pipeline (`media/flow.png`).

### ReadyKit

CONCEPT, NOT FOR FABRICATION    ILLUSTRATIVE, SOFTWARE PROJECT



Grey: Laptop, 14 in class for scale

Figure 1. Illustrative massing of the kit's outputs beside a laptop for scale: document set, PDF house style bands, drawing sheet, printed massing part, TRL badge, guardrail card and template folder.

**ReadyKit: documentation pipeline**

CONCEPT, NOT FOR FABRICATION. ILLUSTRATIVE, SOFTWARE PROJECT. Times measured on one sample repo, 2-core Linux container; other machines will differ.

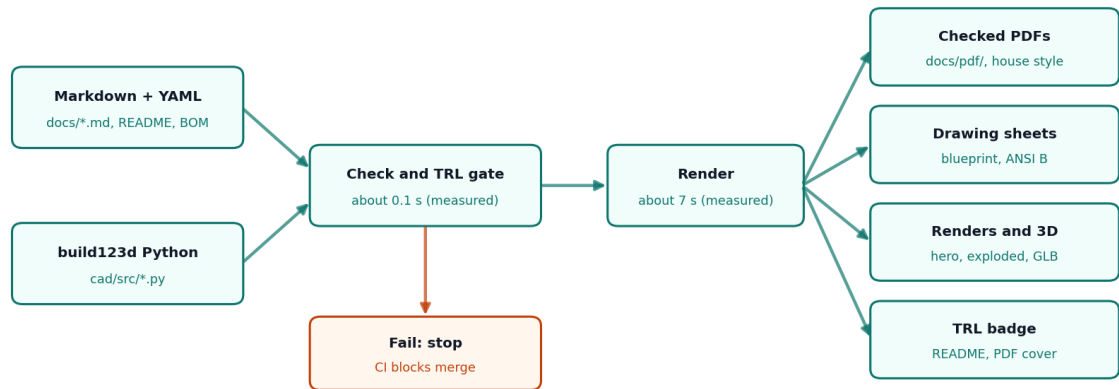


Figure 2. Pipeline from Markdown and build123d inputs through the check and TRL gate to PDFs, drawings, renders and the TRL badge. Times measured on one sample repository.

## ■ How it works

1. **Author in plain text.** Each controlled document is a Markdown file that starts with YAML front matter: document ID, version, status, author, license and revision rows. Geometry lives in `cad/src/*.py` as build123d code.
2. **Check.** `render.py --check` validates every controlled document, then compares the claimed TRL in `project.yaml` with the evidence the standard requires for that level, and applies the portfolio phase cap in `PHASE.yaml`. A failure exits non-zero, so CI blocks the merge.
3. **Render.** `render.py` converts each document to a branded PDF through Python-Markdown, a Jinja2 template and WeasyPrint. The cover carries the document control table, the revision history and the project TRL.
4. **Draw and visualize.** `drawing.py` builds ANSI B sheets with title block and revision table. `concept.py` tessellates a build123d assembly, shades it with a small numpy z-buffer renderer (no GPU, no OpenGL) and writes the hero, exploded and cutaway renders, a blueprint concept sheet, a glTF model and a viewer page.
5. **Guide the agent.** `CLAUDE.md`, the phase file and four slash commands give AI coding agents the same working rules as people: TRL cap, decision rights, one step per session and a review note at the end.

## ■ Main components

Table 1. Modules, numbered as in `bom/bom.csv` and `media/explored.png`.

| No. | Module                       | Current file                                            | State                                         |
|-----|------------------------------|---------------------------------------------------------|-----------------------------------------------|
| 1   | Document control checker     | <code>.kit/render.py</code>                             | Working; 12 of 20 machine-checkable rules     |
| 2   | PDF renderer and house style | <code>.kit/render.py</code> , <code>.kit/style/</code>  | Working; identity hard-coded                  |
| 3   | Drawing sheet generator      | <code>.kit/drawing.py</code>                            | Working; ANSI B only                          |
| 4   | Concept media renderer       | <code>.kit/concept.py</code>                            | Working                                       |
| 5   | TRL gate and badge           | <code>.kit/render.py</code> , <code>PHASE.yaml</code>   | Working for TRL 1 to 6 evidence               |
| 6   | Agent guardrails             | <code>CLAUDE.md</code> , <code>.claude/commands/</code> | Working; decision wording checked by hand     |
| 7   | Template repository and CI   | repo scaffold, <code>.github/workflows/docs.yml</code>  | Exists per repo; not yet a published template |

## ■ First-order numbers

Table 2. Measured and estimated figures. Measurements used kit 1.3.1 on a copy of the TremorTrace repository (3 controlled documents, 6-part massing model), 2-core Linux container, Python 3.11, 2026-09-25.

| Quantity                    | Value                                  | Basis                                              |
|-----------------------------|----------------------------------------|----------------------------------------------------|
| Check time                  | about 0.1 s                            | Measured                                           |
| PDF render, 3 documents     | about 1.6 s                            | Measured                                           |
| Concept media, 6 to 7 parts | about 5 to 7 s                         | Measured (TremorTrace and this repo)               |
| Vendored kit size           | about 1.1 MB, mostly fonts             | Measured                                           |
| Kit source                  | about 810 lines of Python in 3 files   | Measured                                           |
| Portfolio using it          | about 85 project codes in the standard | Counted                                            |
| First-time setup            | 10 to 20 min                           | Estimate; WeasyPrint system libraries dominate     |
| Parts cost                  | \$0                                    | All modules MIT; dependencies free and open source |

Assumption: check time grows roughly linearly with the number of documents, and render time with the number of triangles in the massing model. Neither has been tested beyond the portfolio's own repositories.

## ■ Key design choices

All choices below are **Proposed, awaiting Amish**.

1. **Distribution.** Options: (a) keep the vendored `.kit/` folder copied into each repo; (b) a pip package `readykit` with a small `.kit/` stub pinned to a version; (c) a GitHub template repository plus a sync action.  
Recommendation: (b) plus (c). Pip gives one-command upgrades (R12); the template gives a one-click start.

2. **Identity and branding.** Options: keep portfolio branding; or move organization name, website, repository owner, author and colors into `readykit.yaml`, with Design Molecule values as the default. Recommendation: the config file (R11).
3. **Sheet sizes.** Options: ANSI B only; or add ISO A3 and A2 for teams outside North America. Recommendation: add A3 first.
4. **Command-line interface.** Options: keep separate scripts; or one `readykit` command with `init`, `check`, `render`, `media` and `upgrade`. Recommendation: one command, keeping the scripts as thin wrappers so existing repos keep working.
5. **Licensing of the tool.** The repo carries CERN-OHL-S for hardware and MIT for software. For a software-only project, options: keep both (media and BOM under CERN-OHL-S); or code MIT and documentation CC BY-SA 4.0, matching the standard itself. Recommendation: the second, if Amish wants to change the licenses in `project.yaml`; they are unchanged in this session.
6. **Relationship to existing standards.** Options: stand alone; or export an Open Know-How manifest and a DIN SPEC 3105 documentation checklist from the same front matter. Recommendation: add OKH export at TRL 3, since it costs little and makes projects findable.

## ■ Safety

ReadyKit is software and has no physical hazards of its own. Two risks matter:

**Safety:** A passing check means the documents are complete and consistent with the standard. It does not mean the design is safe, correct or fit for use. Every rendered PDF and drawing must keep the "Draft" or "Not for fabrication" marking until a qualified person has reviewed the engineering.

**Safety:** The checker must never drop the requirement for a safety section in documents that describe mains voltage, heat, pressure, lithium cells, moving machinery or chemicals. Automating that check (Table 3 of RDK-REQ-001) should add a flag, never remove the human review.

## ■ Open questions

- [ ] Distribution route (pip, template, or both)? Proposed, awaiting Amish.
- [ ] Which identity fields must be configurable, and should the Design Molecule brand stay as the default?
- [ ] Should the tool be named ReadyKit on PyPI, and is the name free there and on GitHub?
- [ ] Is Windows native support required, or is WSL2 enough?
- [ ] Which external teams would pilot it (a university lab, an open science hardware group, a startup)?
- [ ] Should the decision-wording check (R13) be generic, or tied to the portfolio's "Proposed, awaiting" phrase?